MÓDULO 1 · S1 · S3 · S6 · SUPERVISADO · CLASIFICACIÓN (TAMBIÉN REGRESIÓN)

Árbol de decisión

`sklearn · DecisionTreeClassifier`

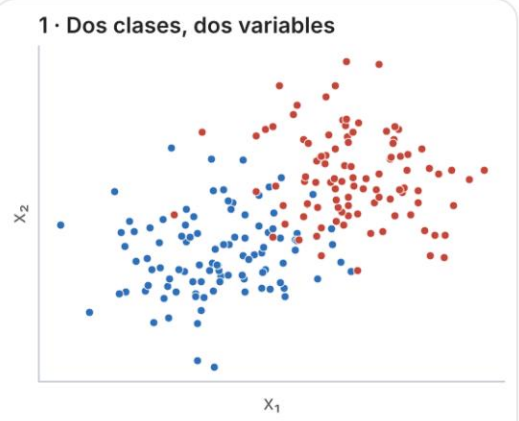
Propósito. Predecir una clase haciendo preguntas de sí/no sobre las variables, una tras otra, hasta llegar a una respuesta.

FUNDAMENTO MÍNIMO

Gini = $1 - \Sigma\, p_k^2$ · gana el corte que más reduce la impureza

La fórmula está para leer las figuras, no al revés. El detalle completo vive en la celda A′ del notebook.

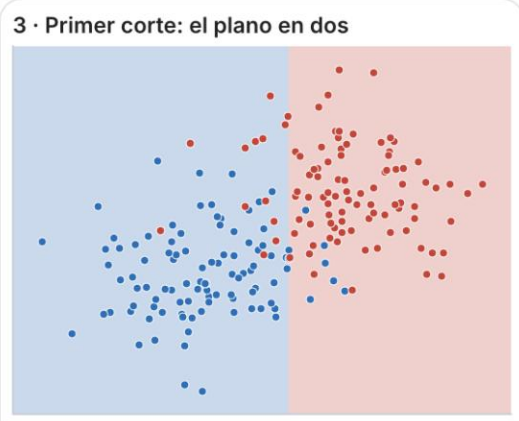
1 CÓMO FUNCIONA — LA HISTORIA EN CUATRO PASOS



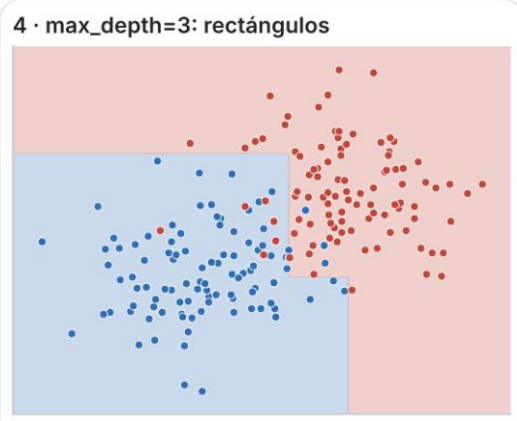
1. Cada punto es una fila con dos variables (x_1, x_2) y una clase (azul / rojo). Queremos reglas que separen los colores.



2. Para cada variable y cada umbral posible, el árbol calcula la impureza Gini de las dos mitades. Elige el corte con el valle más bajo.



3. Un corte = una pregunta («¿$x_1 \leq$ 0,1?»). El plano queda partido en dos rectángulos, cada uno con su clase mayoritaria.



4. Repite la búsqueda dentro de cada mitad. Con más profundidad, más rectángulos... y más riesgo de memorizar el ruido.

2 ASÍ SE VE EN PYTHON · Y QUÉ SIGNIFICA CADA PARÁMETRO

```python
from sklearn.tree import DecisionTreeClassifier

arbol = DecisionTreeClassifier(
    criterion='gini',          # 1
    max_depth=4,               # 2
    min_samples_leaf=20,       # 3
    class_weight='balanced',   # 4
    random_state=42,           # 5
)
arbol.fit(X_train, y_train)
y_pred = arbol.predict(X_test)
```

1. **`criterion`**
   Cómo mide qué tan «mezclada» queda cada mitad. 'gini' (default) o 'entropy'; casi siempre dan el mismo árbol.
2. **`max_depth`**
   Cuántas preguntas seguidas puede hacer. ↑ memoriza (overfitting) · ↓ se queda corto. None = sin límite (S3: 20 403 hojas).
3. **`min_samples_leaf`**
   Mínimo de filas en cada hoja final. ↑ reglas más generales y estables · ↓ hojas de 1 fila que aprenden ruido.
4. **`class_weight`**
   'balanced' da más peso a la clase rara para que el árbol no la ignore (Bank Marketing: 11,7 % de «sí»).
5. **`random_state`**
   Semilla: fija los desempates para que el árbol sea reproducible.

3 CUÁNDO USARLO Y QUÉ VIGILAR

VENTAJAS

- ✔ Se lee como reglas: ideal para explicar al negocio.
- ✔ No necesita escalar las variables.
- ✔ Maneja numéricas y categóricas, y relaciones no lineales.

CUIDADO CON

- ! Sin límite memoriza: train 0,84 vs test 0,74 en S3.
- ! Inestable: un cambio pequeño en los datos cambia todo el árbol.
- ! Aprende tautologías y fugas con gusto (el calendario en S2, `direccion` en S3).

DENTRO DEL `.FIT()`

QUÉ MINIMIZA	Impureza Gini de cada corte
MOVIENDO QUÉ	La variable y el umbral de cada pregunta
CÓMO LO BUSCA	Búsqueda exhaustiva y voraz: prueba todo, se queda con el mejor, nunca vuelve atrás
PERILLA DEL OPTIMIZADOR	Ninguna del optimizador; max_depth es del modelo

EN EL CURSO Incidentes viales Medellín (M1) y Bank Marketing (S6). Trampa: el árbol que aprendió el calendario (S2).

MÓDULO 2 · S4 · SUPERVISADO · REGRESIÓN (Y CONTINUA)

Regresión lineal

`sklearn · LinearRegression`

Propósito. Predecir un número como suma ponderada de las variables: cada coeficiente dice cuánto cambia la y cuando esa variable sube una unidad.

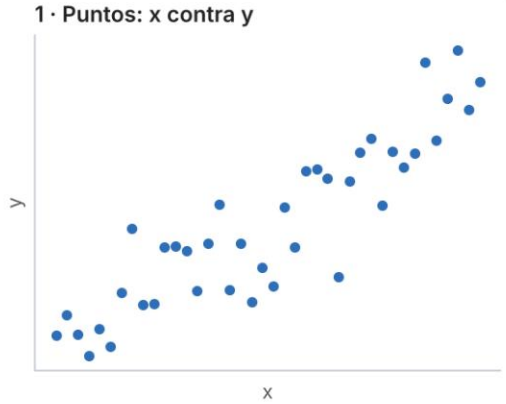
FUNDAMENTO MÍNIMO

$\hat{y} = \beta_0 + \beta_1 x_1 + \ldots + \beta_p x_p$ · minimiza $\Sigma (y_i - \hat{y}_i)^2$

La fórmula está para leer las figuras, no al revés. El detalle completo vive en la celda A′ del notebook.

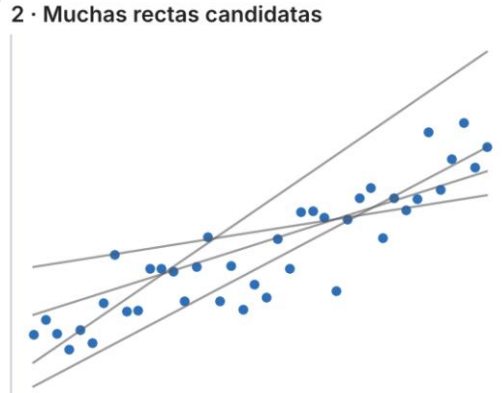
1 CÓMO FUNCIONA — LA HISTORIA EN CUATRO PASOS

1 · Puntos: x contra y



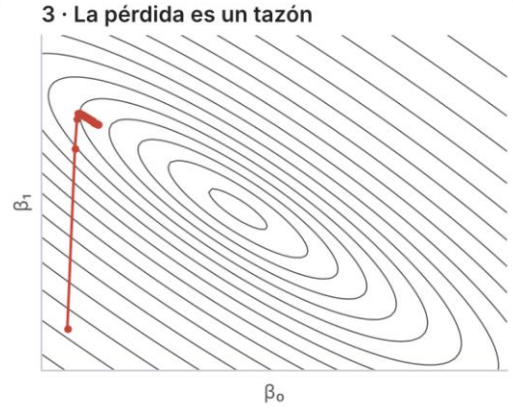
1. Cada punto es una fila: una variable x y la respuesta y (p. ej. edad → costo del seguro).

2 · Muchas rectas candidatas



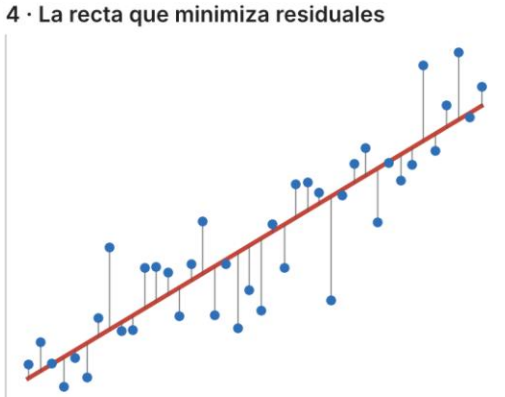
2. Infinitas rectas posibles. ¿Cuál es «la mejor»? La que deja los residuales (distancia vertical punto–recta) más pequeños al cuadrado.

3 · La pérdida es un tazón



3. La suma de cuadrados vista sobre (β_0, β_1) es un tazón convexo: el descenso de gradiente baja paso a paso (η) y la fórmula cerrada salta al fondo directo.

4 · La recta que minimiza residuales



4. La recta final. Cada línea gris es un residual: lo que el modelo no explica. R^2 dice qué fracción de la varianza sí explica.

2 ASÍ SE VE EN PYTHON · Y QUÉ SIGNIFICA CADA PARÁMETRO

```python
from sklearn.linear_model import LinearRegression

modelo = LinearRegression(
    fit_intercept=True,          # 1
)
modelo.fit(X_train, y_train)
modelo.intercept_                # 2
modelo.coef_                     # 3
modelo.score(X_test, y_test)     # 4
```

1. **`fit_intercept`**
   Si estima β_0 (el valor de y cuando todas las x valen 0). Casi siempre True.
2. **`intercept_`**
   β_0 aprendido. En insurance: costo base antes de sumar edad, IMC o fumar.
3. **`coef_`**
   Un β por variable. `smoker_yes` = +23 078 USD: lo que cuesta fumar, manteniendo lo demás fijo.
4. **`score()`**
   R^2 en los datos que le pases. 1 = perfecto (sospechoso: S4), 0 = igual que predecir la media.

3 CUÁNDO USARLO Y QUÉ VIGILAR

VENTAJAS

- ✔ Coeficientes que se leen y se defienden.
- ✔ Rápido, sin hiperparámetros del modelo, con fórmula cerrada.
- ✔ Línea base obligatoria: 70 variantes no le ganaron a OLS.

CUIDADO CON

- ! Solo captura relaciones lineales (o las que tú transformes).
- ! Sensible a valores extremos: un punto raro mueve la recta.
- ! R^2 = 1,000 casi siempre significa que una x define la y (fórmula del ICFES, S4).

DENTRO DEL `.FIT()`

QUÉ MINIMIZA	Suma de residuales al cuadrado
MOVIENDO QUÉ	$\beta_0, \beta_1, \ldots, \beta_p$
CÓMO LO BUSCA	Fórmula cerrada $(X^TX)^{-1}X^Ty$ —o— descenso de gradiente: $\beta \leftarrow \beta - \eta \cdot \nabla L$
PERILLA DEL OPTIMIZADOR	Tasa de aprendizaje η e iteraciones (solo si usas gradiente)

EN EL CURSO Insurance charges → Saber 11 Valle 2022. Trampa: el modelo perfecto (R^2 = 1 con los puntajes por área).

MÓDULO 2 · S4 · SUPERVISADO · REGRESIÓN CON MUCHAS VARIABLES

Ridge y Lasso

Regresión regularizada

`sklearn · Ridge · Lasso`

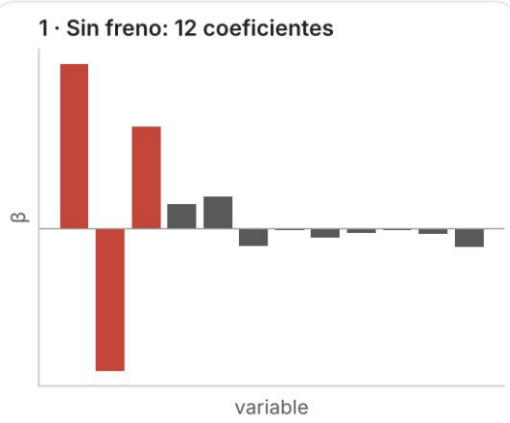
Propósito. La misma regresión lineal, pero con un freno (α) que castiga coeficientes grandes para que el modelo no se aprenda el ruido de decenas de variables.

FUNDAMENTO MÍNIMO

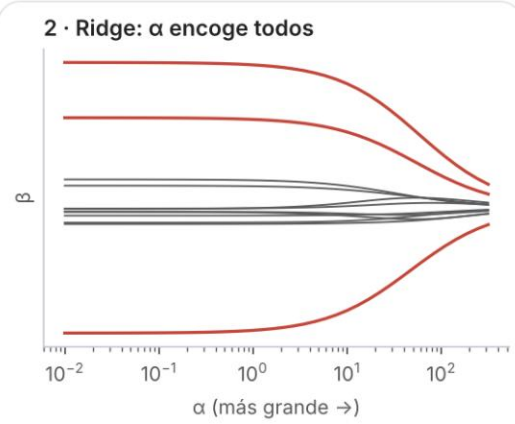
Ridge: $\Sigma(y-\hat{y})^2 + \alpha\ \Sigma\ \beta_j^2$ · Lasso: $\Sigma(y-\hat{y})^2 + \alpha\ \Sigma\ |\beta_j|$

La fórmula está para leer las figuras, no al revés. El detalle completo vive en la celda A′ del notebook.

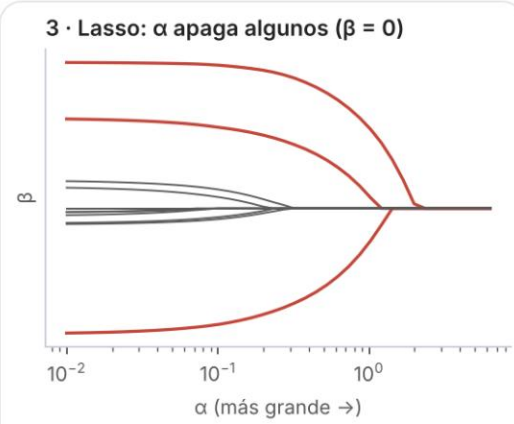
1 CÓMO FUNCIONA — LA HISTORIA EN CUATRO PASOS



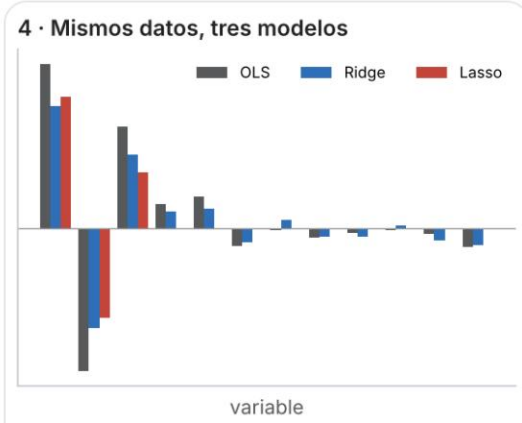
1 Con 12 variables (solo 3 importan, en rojo) OLS reparte coeficientes a todas: las grises son ruido con apariencia de señal.



2 Ridge suma $\alpha\cdot\Sigma\beta^2$ a la pérdida. Al subir α, todos los coeficientes se encogen hacia 0, pero ninguno llega exactamente a 0.



3 Lasso suma $\alpha\cdot\Sigma|\beta|$. Al subir α, los coeficientes inútiles se apagan (β = 0): selecciona variables solo.



4 Mismos datos, tres modelos. Ridge y Lasso recuperan las 3 variables reales y callan el resto; OLS no.

2 ASÍ SE VE EN PYTHON · Y QUÉ SIGNIFICA CADA PARÁMETRO

```python
from sklearn.linear_model import Ridge, Lasso
from sklearn.preprocessing import StandardScaler

X_tr = StandardScaler().fit_transform(X_train)   # 1
ridge = Ridge(alpha=10)                          # 2
lasso = Lasso(alpha=0.5, max_iter=5000)          # 3
ridge.fit(X_tr, y_train)
(lasso.coef_ == 0).sum()                         # 4
```

1 **`StandardScaler`**
Obligatorio antes: la penalización trata igual a todos los β, así que las variables deben estar en la misma escala.

2 **`alpha (Ridge)`**
Fuerza del freno. 0 = OLS. ↑ coeficientes más pequeños, menos varianza, más sesgo. Se elige con validación (S10).

3 **`alpha (Lasso) · max_iter`**
Igual, pero ↑ apaga variables. Lasso no tiene fórmula cerrada: itera por coordenadas; max_iter evita el ConvergenceWarning.

4 **`coef_ == 0`**
Cuántas variables apagó Lasso. Es tu selección automática de variables.

3 CUÁNDO USARLO Y QUÉ VIGILAR

VENTAJAS

- ✔ Controla el overfitting cuando hay más variables que señal.
- ✔ Lasso deja un modelo corto y explicable.
- ✔ Ridge es estable aunque las x estén correlacionadas.

CUIDADO CON

- ! Sin escalar, α castiga más a las variables con unidades pequeñas.
- ! α se elige con CV, nunca mirando el test.
- ! Los coeficientes ya no se leen como «efecto puro»: están encogidos a propósito.

DENTRO DEL `.FIT()`

QUÉ MINIMIZA	Suma de cuadrados + penalización $\alpha\cdot\Sigma\beta^2$ (Ridge) o $\alpha\cdot\Sigma\|\beta\|$ (Lasso)
MOVIENDO QUÉ	$\beta_0, \beta_1, \ldots, \beta_p$
CÓMO LO BUSCA	Ridge: fórmula cerrada · Lasso: descenso por coordenadas
PERILLA DEL OPTIMIZADOR	α es del modelo; la tolerancia y max_iter son del optimizador

EN EL CURSO Saber 11 Valle 2022 con decenas de dummies (S4). Se retoma dentro del Pipeline en S10–S11.

MÓDULO 2 · S5 · SUPERVISADO · CLASIFICACIÓN BINARIA

Regresión logística

`sklearn · LogisticRegression`

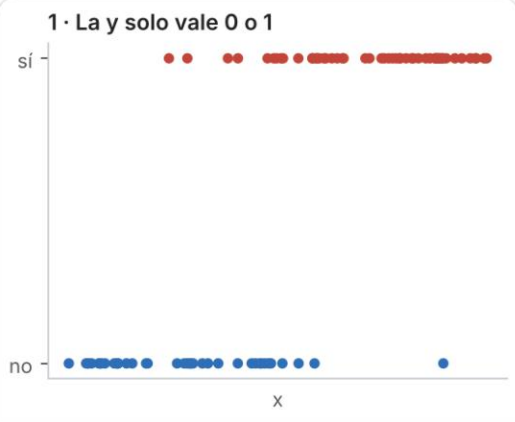
Propósito. Estimar la probabilidad de que una fila sea «sí» pasando una suma lineal por la sigmoide; luego tú decides el umbral que convierte esa probabilidad en decisión.

FUNDAMENTO MÍNIMO

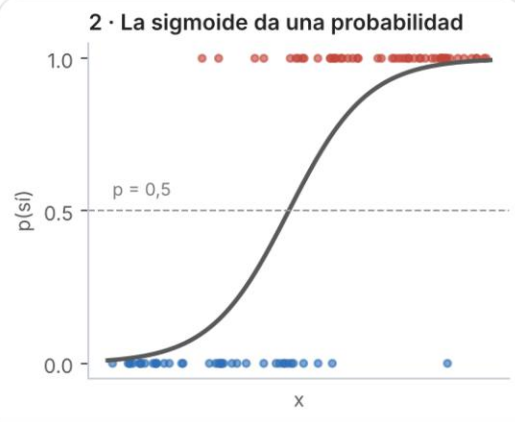
$p = 1 / (1 + e^{(-z)})$, $z = \beta_0 + \beta_1 x_1 + \ldots$ · frontera en $p = 0{,}5$

La fórmula está para leer las figuras, no al revés. El detalle completo vive en la celda A′ del notebook.

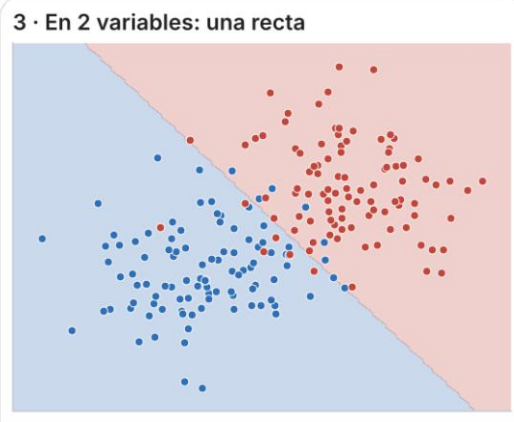
1 CÓMO FUNCIONA — LA HISTORIA EN CUATRO PASOS



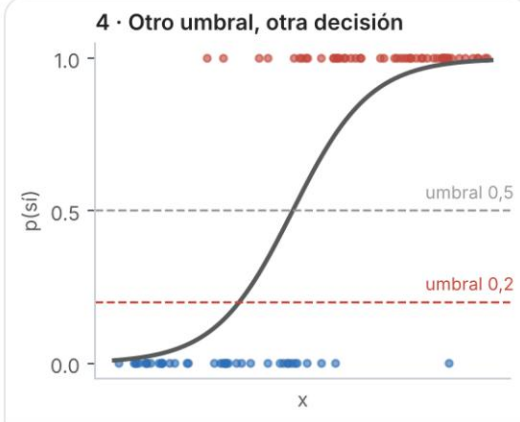
1. La y es binaria (depósito sí/no). Una recta no sirve: predeciría valores fuera de 0 y 1.



2. La sigmoide aplasta cualquier z al rango (0, 1). El resultado se lee como probabilidad de «sí».



3. Con dos variables, p = 0,5 dibuja una recta: la frontera de decisión. Es un modelo lineal, solo que en la escala del logit.



4. Bajar el umbral (0,5 → 0,2) encuentra más «sí» (recall sube) a costa de más falsas alarmas. La decisión es de negocio, no del algoritmo.

2 ASÍ SE VE EN PYTHON · Y QUÉ SIGNIFICA CADA PARÁMETRO

```python
from sklearn.linear_model import LogisticRegression

log = LogisticRegression(
    C=1.0,                          # 1
    class_weight='balanced',        # 2
    max_iter=1000,                  # 3
)
log.fit(X_train_esc, y_train)
p = log.predict_proba(X_test_esc)[:, 1]   # 4
y_pred = (p >= 0.2).astype(int)           # 5
```

1. **`C`**
   Inverso de la regularización (C = 1/α). ↓ C = más freno sobre los β · ↑ C = más libre. Escala las x antes.
2. **`class_weight`**
   'balanced' pesa más la clase rara. En Bank Marketing sube el recall de 0,18 a 0,62.
3. **`max_iter`**
   No hay fórmula cerrada: lbfgs itera. Si ves ConvergenceWarning, sube max_iter o escala las x.
4. **`predict_proba`**
   La probabilidad, no la clase. Es lo que debes guardar: la clase depende del umbral.
5. **`umbral`**
   No es un parámetro de sklearn: lo eliges tú con la matriz de confusión y los costos (20 000 COP/llamada vs 100 000 COP/depósito).

3 CUÁNDO USARLO Y QUÉ VIGILAR

VENTAJAS

- ✔ Entrega probabilidades calibradas, no solo etiquetas.
- ✔ Coeficientes interpretables: e^β es el odds ratio.
- ✔ Rápida y difícil de sobreajustar con pocas variables.

CUIDADO CON

- ! Frontera lineal: no captura interacciones sin ayuda.
- ! Accuracy miente con clases desbalanceadas (0,894 vs 0,883 del tonto).
- ! Necesita variables escaladas y sin fugas (`duration`, S6).

DENTRO DEL `.FIT()`

QUÉ MINIMIZA	Log-loss (log-verosimilitud negativa)
MOVIENDO QUÉ	$\beta_0, \beta_1, \ldots, \beta_p$
CÓMO LO BUSCA	Iterativo: lbfgs, un primo del descenso de gradiente que estima la curvatura
PERILLA DEL OPTIMIZADOR	max_iter y tolerancia; C es del modelo

EN EL CURSO Bank Marketing (S5). Trampa: accuracy engañosa — 0,894 con recall 0,18.

MÓDULO 2 · S5 · SUPERVISADO · CLASIFICACIÓN (TAMBIÉN REGRESIÓN)

k vecinos más cercanos

k-NN

`sklearn · KNeighborsClassifier`

Propósito. Clasificar una fila nueva mirando las k filas de entrenamiento más parecidas y copiando la mayoría. No aprende reglas: memoriza y compara.

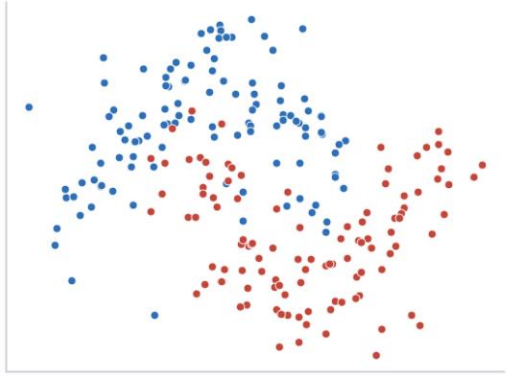
FUNDAMENTO MÍNIMO

$d(x, x') = \sqrt{\Sigma\,(x_j - x'_j)^2}$ · vota la mayoría de los k vecinos

La fórmula está para leer las figuras, no al revés. El detalle completo vive en la celda A′ del notebook.

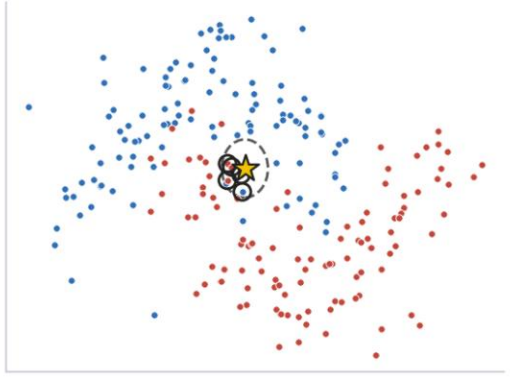
1 CÓMO FUNCIONA — LA HISTORIA EN CUATRO PASOS

1 · Memoriza todos los puntos



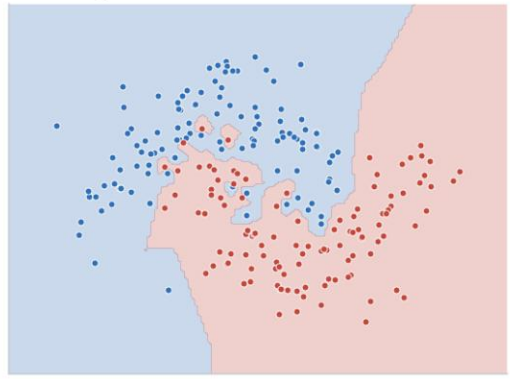
1 El `.fit()` no calcula nada: guarda la tabla completa. Todo el trabajo ocurre al predecir.

2 · Punto nuevo: sus k=5 vecinos votan



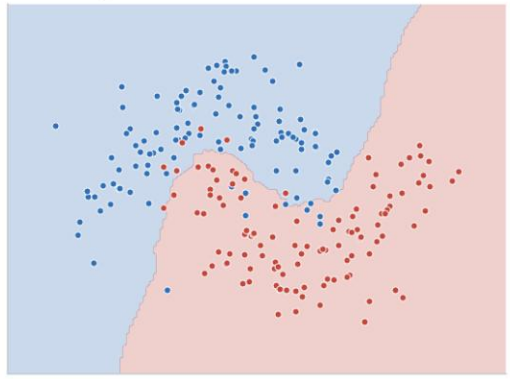
2 Llega un punto nuevo (estrella). Se calcula su distancia a todos los demás y se toman los k = 5 más cercanos: 3 rojos, 2 azules → rojo.

3 · n_neighbors=1: memoriza



3 Con k = 1 cada punto manda sobre su vecindad: frontera dentada, accuracy de train = 1,0. Puro overfitting.

4 · n_neighbors=25: suaviza



4 Con k = 25 el voto se promedia: frontera suave que ignora puntos aislados. Demasiado k y todo se vuelve la clase mayoritaria.

2 ASÍ SE VE EN PYTHON · Y QUÉ SIGNIFICA CADA PARÁMETRO

```python
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler

esc = StandardScaler().fit(X_train)                    # 1
knn = KNeighborsClassifier(
    n_neighbors=15,                                    # 2
    weights='distance',                                # 3
    metric='euclidean',                                # 4
)
knn.fit(esc.transform(X_train), y_train)
knn.predict(esc.transform(X_test))                     # 5
```

1 **`StandardScaler`**
Sin escalar, la variable con números más grandes (balance en pesos) decide sola la distancia. En S5: recall 0,20 → 0,03.

2 **`n_neighbors`**
Cuántos vecinos votan. ↓ memoriza (k=1) · ↑ suaviza hasta ignorar todo. Usa impares para evitar empates.

3 **`weights`**
'uniform': cada vecino un voto. 'distance': los más cercanos pesan más.

4 **`metric`**
Cómo se mide «parecido». Euclidiana por defecto; 'manhattan' suma diferencias absolutas.

5 **`predict`**
Aquí está el costo: compara contra todas las filas de entrenamiento cada vez.

3 CUÁNDO USARLO Y QUÉ VIGILAR

VENTAJAS

- ✔ Casi sin supuestos: aprende fronteras de cualquier forma.
- ✔ Trivial de explicar: «se parece a estos».
- ✔ Un solo hiperparámetro que importa de verdad (k).

CUIDADO CON

- ! Lento al predecir con muchas filas.
- ! Pierde sentido con muchas variables (todo queda lejos de todo).
- ! Depende por completo de la escala y de la métrica.

DENTRO DEL `.FIT()`

QUÉ MINIMIZA	Nada
MOVIENDO QUÉ	Nada
CÓMO LO BUSCA	No optimiza: memoriza las filas y compara al predecir
PERILLA DEL OPTIMIZADOR	k es del modelo; no hay perilla del optimizador

EN EL CURSO Bank Marketing (S5). Trampa: k-NN sin escalar — balance en pesos decide los vecinos.

MÓDULO 2 · S6 · SUPERVISADO · CLASIFICACIÓN Y REGRESIÓN

Bosque aleatorio

Random Forest

`sklearn · RandomForestClassifier`

Propósito. Entrenar cientos de árboles distintos, cada uno con una muestra y variables al azar, y dejar que voten. El promedio corrige lo que un árbol solo memoriza.

FUNDAMENTO MÍNIMO

$\hat{y}$ = promedio (o voto) de B árboles entrenados sobre muestras bootstrap

La fórmula está para leer las figuras, no al revés. El detalle completo vive en la celda A′ del notebook.

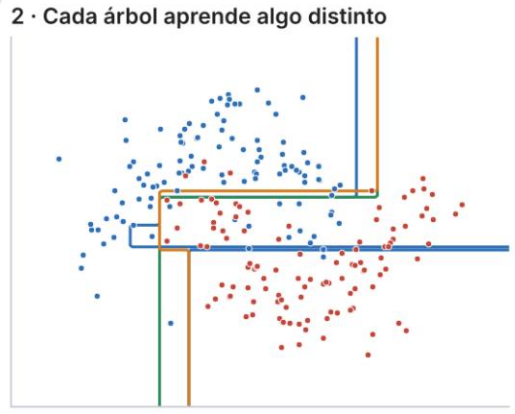
1 CÓMO FUNCIONA — LA HISTORIA EN CUATRO PASOS

1 · Muestras bootstrap distintas



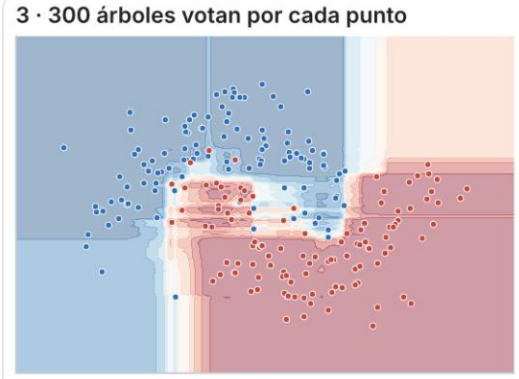
1. Bootstrap: cada árbol recibe una muestra con reemplazo de las filas. Ningún árbol ve exactamente los mismos datos.

2 · Cada árbol aprende algo distinto



2. Además, en cada corte solo puede mirar un subconjunto de variables (max_features). Resultado: árboles distintos que se equivocan en lugares distintos.

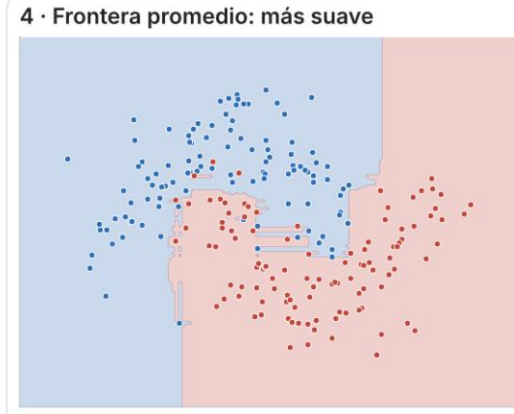
3 · 300 árboles votan por cada punto



color = % de árboles que dicen «sí»

3. Para predecir, cada árbol vota. El color es el porcentaje de árboles que dicen «sí»: eso es predict_proba.

4 · Frontera promedio: más suave



4. La frontera final es suave: el árbol sin límite daba AUC 0,61 en S6; el bosque sin límite ~0,79.

2 ASÍ SE VE EN PYTHON · Y QUÉ SIGNIFICA CADA PARÁMETRO

```python
from sklearn.ensemble import RandomForestClassifier

bosque = RandomForestClassifier(
    n_estimators=300,          # 1
    max_depth=None,            # 2
    max_features='sqrt',       # 3
    min_samples_leaf=5,        # 4
    class_weight='balanced',   # 5
    n_jobs=-1, random_state=42,
)
bosque.fit(X_train, y_train)
bosque.feature_importances_    # 6
```

1. **`n_estimators`** — Número de árboles. ↑ más estable y más lento; a partir de ~200–500 ya no mejora.
2. **`max_depth`** — Profundidad de cada árbol. None deja crecer; el promedio compensa el overfitting individual.
3. **`max_features`** — Variables candidatas en cada corte. 'sqrt' = √p: obliga a los árboles a ser diferentes.
4. **`min_samples_leaf`** — Filas mínimas por hoja: ↑ árboles más simples y predicciones más suaves.
5. **`class_weight`** — 'balanced' sube el recall de la clase rara (S6: recall 0,54, F1 0,48).
6. **`feature_importances_`** — Cuánto redujo la impureza cada variable, sumado en todos los árboles. Ojo: premia variables con muchos valores.

3 CUÁNDO USARLO Y QUÉ VIGILAR

VENTAJAS

- ✔ Funciona bien casi sin ajustar nada.
- ✔ Robusto a ruido y valores extremos; no necesita escalar.
- ✔ Da importancias de variables y probabilidades.

CUIDADO CON

- ! Caja negra: 300 árboles no se leen como reglas.
- ! Más lento y pesado que un árbol o una logística.
- ! Las importancias no son causalidad y adoran las fugas (`duration`: AUC 0,80 → 0,93).

DENTRO DEL `.FIT()`

QUÉ MINIMIZA	Gini en cada corte de cada árbol
MOVIENDO QUÉ	Los cortes de cada árbol, por separado
CÓMO LO BUSCA	Muchos árboles voraces sobre muestras bootstrap; no hay optimización global
PERILLA DEL OPTIMIZADOR	Ninguna

EN EL CURSO Bank Marketing (S6, base de E2). Trampa: `duration`, la columna del futuro.

MÓDULO 3 · S7 · S9 · NO SUPERVISADO · CLUSTERING (SIN Y)

K-means

sklearn · KMeans

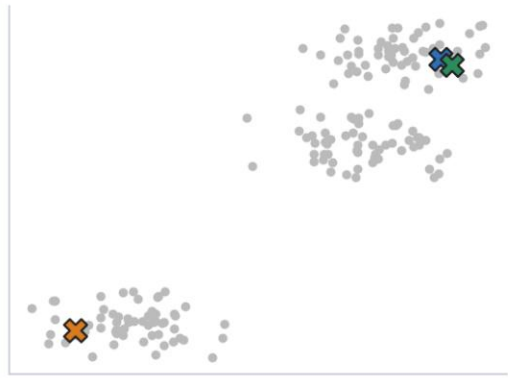
Propósito. Agrupar filas parecidas en k grupos compactos, moviendo k centroides hasta que cada punto quede con el más cercano. Tú eliges k; el algoritmo no lo sabe.

FUNDAMENTO MÍNIMO

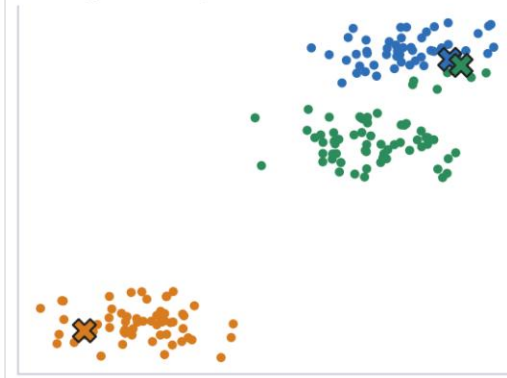
minimiza $\Sigma_k \Sigma_{i \in C_k} \|x_i - \mu_k\|^2$ · asignar → recalcular → repetir

La fórmula está para leer las figuras, no al revés. El detalle completo vive en la celda A′ del notebook.

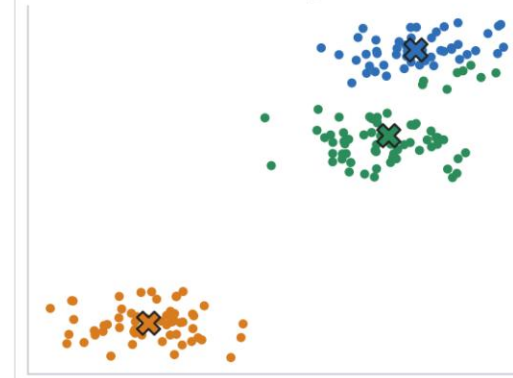
1 CÓMO FUNCIONA — LA HISTORIA EN CUATRO PASOS



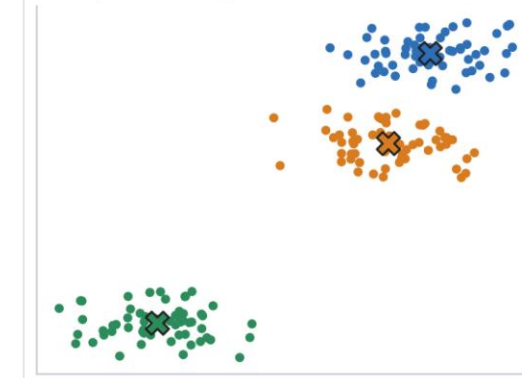
1 Se eligen k = 3 centroides (X) al azar entre los puntos. No hay etiquetas: nadie dice cuál grupo es correcto.



2 Asignar: cada punto se pinta del color del centroide más cercano (distancia euclidiana).



3 Recalcular: cada centroide salta al promedio de sus puntos. Los centroides se mueven hacia el corazón de cada grupo.



4 Se repite hasta que ninguna asignación cambie. Llega a un mínimo local: otra semilla puede dar otro resultado (por eso n_init).

2 ASÍ SE VE EN PYTHON · Y QUÉ SIGNIFICA CADA PARÁMETRO

```python
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

X_esc = StandardScaler().fit_transform(X)                    # 1
km = KMeans(
    n_clusters=4,                                            # 2
    n_init=10,                                               # 3
    max_iter=300,                                            # 4
    random_state=0,
)
etiquetas = km.fit_predict(X_esc)
km.inertia_                                                  # 5
```

1. **`StandardScaler`**
   Sin escalar, agrupa por unidad de medida: en Penguins, los gramos de masa mandan sobre los milímetros de pico.
2. **`n_clusters`**
   k: cuántos grupos quieres. Se tantea con el codo de inertia_ o con silhouette, y se valida con sentido de negocio.
3. **`n_init`**
   Cuántas veces arranca con centroides distintos; se queda con la mejor inercia. Protege del mínimo local.
4. **`max_iter`**
   Tope de rondas asignar→recalcular por arranque. Suele converger mucho antes.
5. **`inertia_`**
   Suma de distancias² al centroide: lo que K-means minimiza. Siempre baja al subir k, por eso solo sirve para buscar el codo.

3 CUÁNDO USARLO Y QUÉ VIGILAR

VENTAJAS

- ✔ Simple, rápido y escala a millones de filas.
- ✔ Centroides interpretables: el «perfil promedio» de cada grupo.
- ✔ Base para segmentar clientes, municipios, productos.

CUIDADO CON

- ! Tienes que fijar k antes.
- ! Busca grupos redondos y de tamaño parecido; falla con formas raras.
- ! Sensible a escala, valores extremos y semilla.

DENTRO DEL `.FIT()`

QUÉ MINIMIZA	Suma de distancias² de cada punto a su centroide (inercia)
MOVIENDO QUÉ	Las asignaciones y los centroides
CÓMO LO BUSCA	Alternancia: asignar → recalcular; converge a un mínimo local
PERILLA DEL OPTIMIZADOR	n_init y max_iter

EN EL CURSO make_blobs → Penguins → municipios DANE (S7, S9). Trampa: K-means sin escalar agrupa por unidad de medida.

MÓDULO 3 · S7 · NO SUPERVISADO · CLUSTERING (SIN Y)

DBSCAN

Clustering por densidad

`sklearn · DBSCAN`

Propósito. Encontrar grupos como zonas densas de puntos, de cualquier forma, y marcar como ruido lo que queda aislado. No hay que decir cuántos grupos hay.

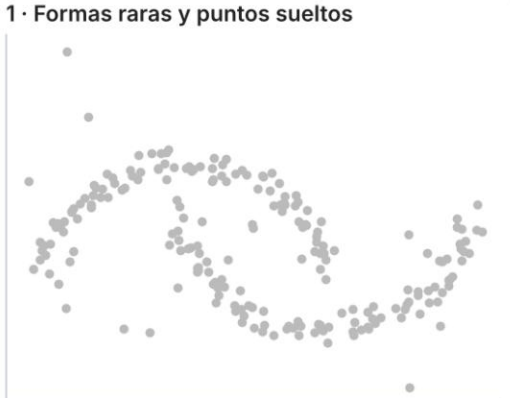
FUNDAMENTO MÍNIMO

núcleo: ≥ min_samples vecinos a distancia ≤ ε · borde: cerca de un núcleo · ruido: nada

La fórmula está para leer las figuras, no al revés. El detalle completo vive en la celda A′ del notebook.

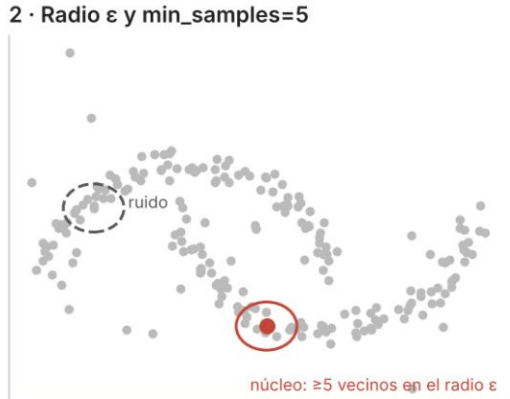
1 CÓMO FUNCIONA — LA HISTORIA EN CUATRO PASOS

1 · Formas raras y puntos sueltos



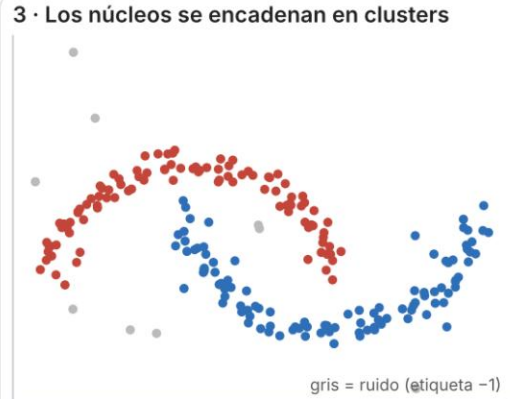
1 Dos medialunas y puntos sueltos. K-means partiría las lunas por la mitad; aquí la forma no importa, la densidad sí.

2 · Radio ε y min_samples=5



2 Para cada punto se dibuja un círculo de radio ε. Si adentro hay ≥ min_samples puntos, es núcleo. Si no, es borde (si toca un núcleo) o ruido.

3 · Los núcleos se encadenan en clusters



3 Los núcleos vecinos se encadenan: un cluster es todo lo que se alcanza saltando de núcleo en núcleo. Los sueltos reciben la etiqueta −1.

4 · ε grande: todo se junta



4 ε demasiado grande une todo en un cluster; demasiado pequeño deja casi todo como ruido. ε se tantea mirando las distancias al k-ésimo vecino.

2 ASÍ SE VE EN PYTHON · Y QUÉ SIGNIFICA CADA PARÁMETRO

```python
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler

X_esc = StandardScaler().fit_transform(X)          # 1
db = DBSCAN(
    eps=0.5,                                       # 2
    min_samples=5,                                 # 3
    metric='euclidean',                            # 4
)
etiquetas = db.fit_predict(X_esc)
(etiquetas == -1).mean()                           # 5
```

1 **`StandardScaler`**
ε es una distancia: solo tiene sentido si todas las variables están en la misma escala.

2 **`eps (ε)`**
Radio del vecindario. Es la perilla crítica: ↑ clusters más grandes y menos ruido · ↓ más ruido y clusters fragmentados.

3 **`min_samples`**
Vecinos mínimos para ser núcleo. ↑ exige zonas más densas; regla práctica: ≥ número de variables + 1.

4 **`metric`**
Cómo se mide la distancia dentro del radio.

5 **`etiquetas == −1`**
Fracción de ruido. Si es > 20–30 %, ε es muy pequeño o los datos no tienen zonas densas.

3 CUÁNDO USARLO Y QUÉ VIGILAR

VENTAJAS

- ✔ No pide el número de clusters.
- ✔ Encuentra formas arbitrarias (anillos, lunas, calles).
- ✔ Detecta ruido y valores atípicos de paso.

CUIDADO CON

- ! Un solo ε para todo: falla si los grupos tienen densidades distintas.
- ! Difícil de calibrar en muchas dimensiones.
- ! No da centroides ni predice puntos nuevos directamente.

DENTRO DEL `.FIT()`

QUÉ MINIMIZA	Nada: no hay función de pérdida
MOVIENDO QUÉ	Nada
CÓMO LO BUSCA	Recorre vecindades y encadena núcleos
PERILLA DEL OPTIMIZADOR	ε y min_samples son del modelo; no hay optimizador

EN EL CURSO make_blobs y municipios DANE (S7). Contraste con K-means: las formas que K-means no puede ver.

MÓDULO 3 · S8 · NO SUPERVISADO · REDUCCIÓN DE DIMENSIONALIDAD

PCA

Análisis de componentes principales

`sklearn · PCA`

Propósito. Resumir muchas variables correlacionadas en pocas direcciones nuevas (componentes) que conservan la mayor varianza posible, para ver y modelar con menos columnas.

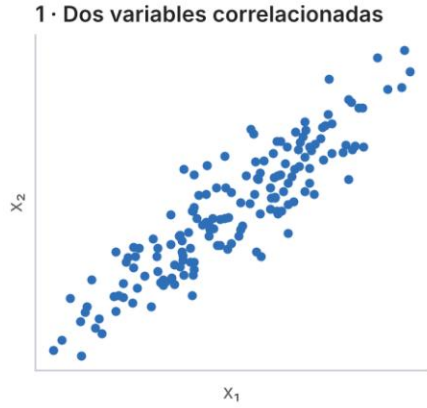
FUNDAMENTO MÍNIMO

componentes = autovectores de la covarianza · varianza explicada = λ_k / $\Sigma\lambda$

La fórmula está para leer las figuras, no al revés. El detalle completo vive en la celda A′ del notebook.

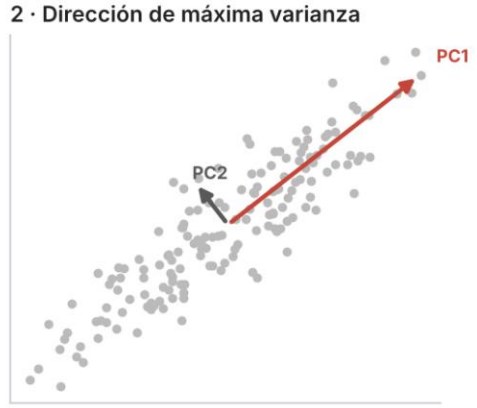
1 CÓMO FUNCIONA — LA HISTORIA EN CUATRO PASOS

1 · Dos variables correlacionadas



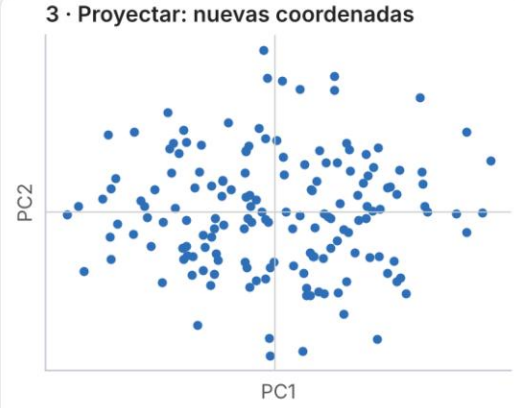
1 Dos variables que se mueven juntas (p. ej. NBI y ruralidad). Gran parte de la información está repetida.

2 · Dirección de máxima varianza



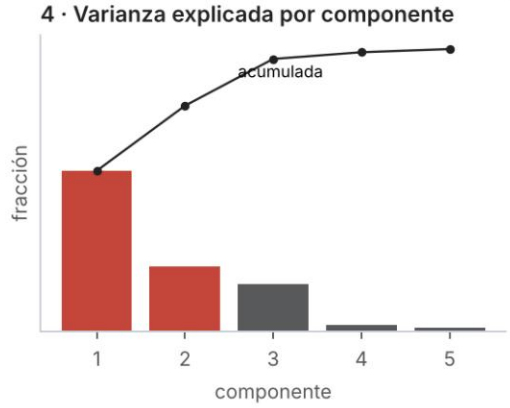
2 PCA busca la dirección en la que los datos se estiran más (PC1) y luego la perpendicular (PC2). Son rotaciones, no selecciones.

3 · Proyectar: nuevas coordenadas



3 Proyectar = reescribir cada fila en esas direcciones. PC1 concentra casi toda la dispersión; PC2 casi nada.

4 · Varianza explicada por componente



4 Con más variables, la gráfica de varianza explicada dice cuántos componentes vale la pena conservar (aquí: 2 de 5 explican ~80 %).

2 ASÍ SE VE EN PYTHON · Y QUÉ SIGNIFICA CADA PARÁMETRO

```python
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

X_esc = StandardScaler().fit_transform(X)   # 1
pca = PCA(n_components=2)                   # 2
Z = pca.fit_transform(X_esc)
pca.explained_variance_ratio_               # 3
pca.components_                             # 4
```

1 **StandardScaler**
Sin escalar, la variable con mayor varianza numérica (población) se vuelve PC1 ella sola.

2 **n_components**
Cuántas direcciones conservar: un entero (2 para graficar) o una fracción (0.9 = las necesarias para explicar el 90 %).

3 **explained_variance_ratio_**
Fracción de la varianza total que captura cada componente. Se grafica acumulada para decidir cuántos quedarse.

4 **components_**
Los pesos de cada variable original en cada componente: así se lee qué significa PC1 («desarrollo urbano», p. ej.).

3 CUÁNDO USARLO Y QUÉ VIGILAR

VENTAJAS

- ✔ Elimina redundancia y ruido antes de agrupar o modelar.
- ✔ Permite ver en 2-D datos de 20 variables.
- ✔ Determinista: mismo resultado siempre (fórmula cerrada).

CUIDADO CON

- ! Los componentes mezclan variables: cuesta explicarlos.
- ! Solo captura relaciones lineales.
- ! La varianza no es importancia: lo que más varía no es siempre lo que más importa.

DENTRO DEL `.FIT()`

QUÉ MINIMIZA	Maximiza la varianza explicada por cada dirección
MOVIENDO QUÉ	Las direcciones (autovectores)
CÓMO LO BUSCA	Descomposición de la matriz de covarianza: fórmula cerrada
PERILLA DEL OPTIMIZADOR	Ninguna

EN EL CURSO Penguins y municipios DANE (S8). Base de E3 junto con K-means.

MÓDULO 3 · S8 · NO SUPERVISADO · REDUCCIÓN A 2-D PARA VISUALIZAR

t-SNE

Visualización no lineal

`sklearn · TSNE`

Propósito. Dibujar en 2-D datos de muchas variables conservando quién es vecino de quién. Sirve para mirar, no para medir distancias ni para modelar.

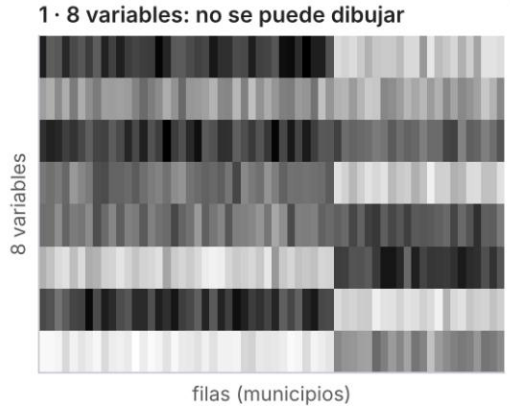
FUNDAMENTO MÍNIMO

minimiza KL(P ‖ Q): P = vecindades en alta dimensión, Q = vecindades en el mapa 2-D

La fórmula está para leer las figuras, no al revés. El detalle completo vive en la celda A′ del notebook.

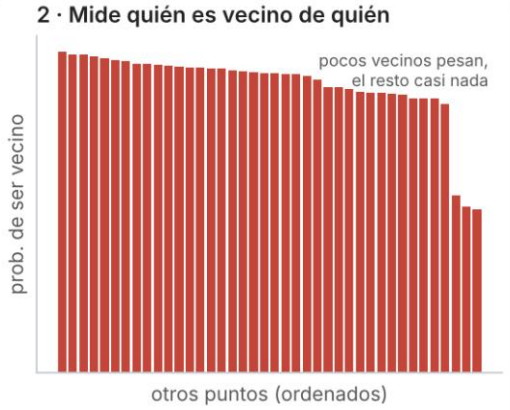
1 CÓMO FUNCIONA — LA HISTORIA EN CUATRO PASOS

1 · 8 variables: no se puede dibujar



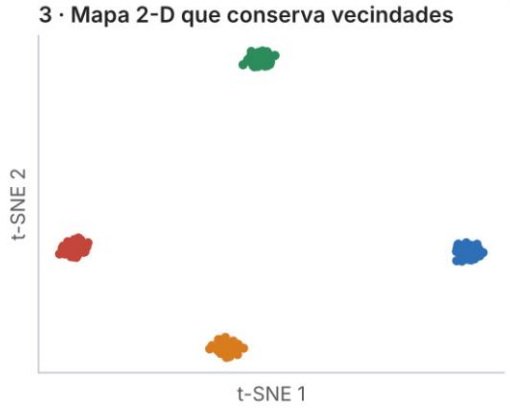
1 Cada fila tiene 8 variables: imposible de graficar. Buscamos un mapa plano que respete la estructura.

2 · Mide quién es vecino de quién



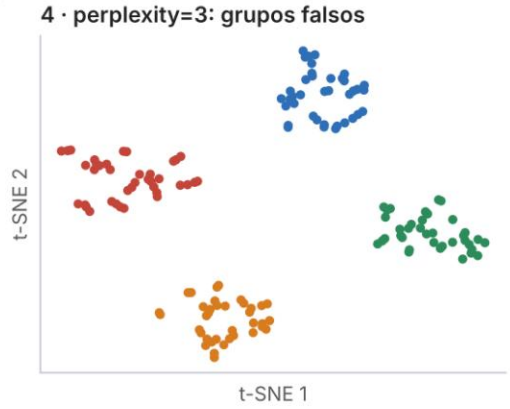
2 Para cada punto se calcula la probabilidad de que cada otro punto sea su vecino (gaussiana). perplexity fija cuántos vecinos «cuentan».

3 · Mapa 2-D que conserva vecindades



3 Se colocan los puntos en 2-D al azar y se mueven por descenso de gradiente hasta que las vecindades del mapa imiten las originales.

4 · perplexity=3: grupos falsos



4 Con perplexity muy baja aparecen grupos que no existen. Cambia la semilla o la perplexity y el mapa cambia: los clusters bonitos no son evidencia.

2 ASÍ SE VE EN PYTHON · Y QUÉ SIGNIFICA CADA PARÁMETRO

```python
from sklearn.manifold import TSNE

tsne = TSNE(
    n_components=2,
    perplexity=30,          # 1
    learning_rate='auto',   # 2
    max_iter=1000,          # 3
    init='pca',             # 4
    random_state=0,         # 5
)
Z = tsne.fit_transform(X_esc)
```

1. **perplexity**
   Número efectivo de vecinos que cuentan (5–50). ↓ grupos pequeños y falsos · ↑ estructura global, grupos difusos.
2. **learning_rate**
   Tamaño del paso del descenso de gradiente. 'auto' lo ajusta al tamaño de los datos; muy alto deja una «bola».
3. **max_iter**
   Iteraciones del descenso. Si el mapa se ve a medio formar, sube.
4. **init**
   'pca' arranca desde PCA: más estable y reproducible que 'random'.
5. **random_state**
   La pérdida no es convexa: cada semilla da un mapa distinto. Corre varias antes de creer un cluster.

3 CUÁNDO USARLO Y QUÉ VIGILAR

VENTAJAS

- ✔ Revela estructura que PCA (lineal) no muestra.
- ✔ Muy bueno para explorar grupos antes de etiquetarlos.
- ✔ Estándar para mirar embeddings e imágenes.

CUIDADO CON

- ! Las distancias y tamaños en el mapa no significan nada.
- ! No tiene `.transform()`: no sirve para datos nuevos.
- ! Lento con muchas filas y fácil de sobreinterpretar.

DENTRO DEL `.FIT()`

QUÉ MINIMIZA	Divergencia KL entre vecindades originales y del mapa
MOVIENDO QUÉ	Las posiciones 2-D de cada punto
CÓMO LO BUSCA	Descenso de gradiente no convexo: resultado distinto por semilla
PERILLA DEL OPTIMIZADOR	learning_rate, perplexity, max_iter

EN EL CURSO Municipios DANE (S8). Trampa: clusters «bonitos» que desaparecen al cambiar semilla o perplexity.

MÓDULO 4 · S11 · SUPERVISADO · CLASIFICACIÓN Y REGRESIÓN EN TABLAS

Gradient boosting

`sklearn · HistGradientBoostingClassifier`

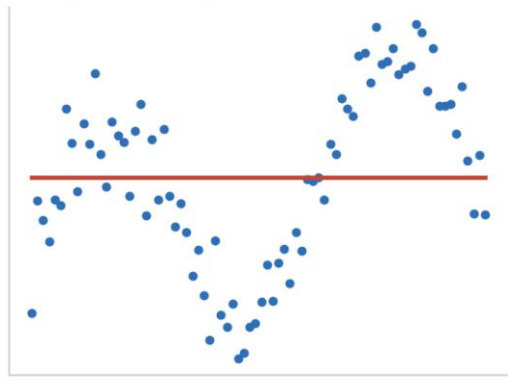
Propósito. Construir el modelo por etapas: cada árbol pequeño nuevo corrige los errores que dejaron los anteriores. Es el modelo de referencia actual para datos tabulares.

FUNDAMENTO MÍNIMO

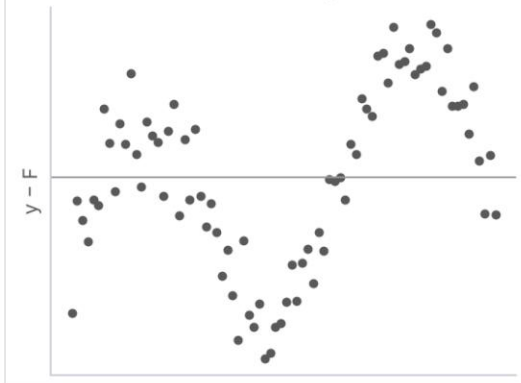
$F_m = F_{m-1} + \eta \cdot h_m$ · cada h_m ajusta los residuales (gradiente) de F_{m-1}

La fórmula está para leer las figuras, no al revés. El detalle completo vive en la celda A′ del notebook.

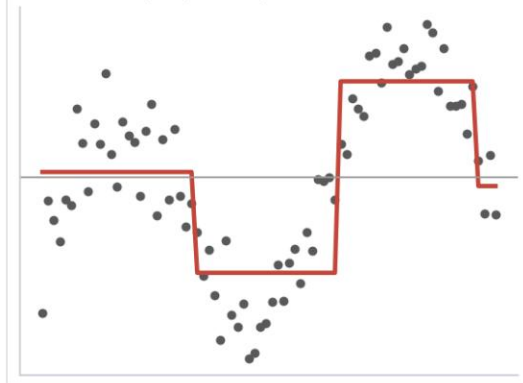
1 CÓMO FUNCIONA — LA HISTORIA EN CUATRO PASOS



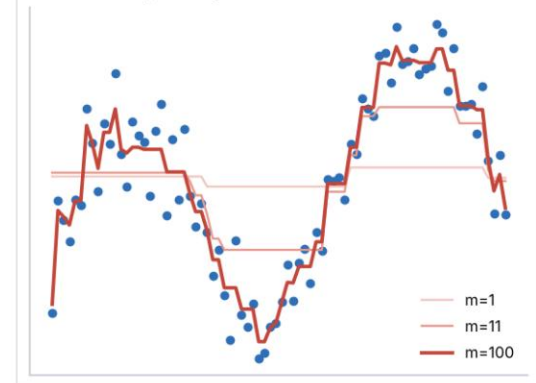
1 F_0 es una predicción tonta: el promedio (o la proporción de «sí»). Todo lo demás se construye encima.



2 Residual = y − F. Es lo que el modelo todavía no explica; tiene forma, no es ruido.



3 Un árbol pequeño (max_depth 2–3) aprende la forma del residual. No intenta predecir y: predice el error.



4 F se actualiza sumando η·árbol. 100 rondas de árboles débiles dan una curva precisa. η pequeño + más árboles = más seguro.

2 ASÍ SE VE EN PYTHON · Y QUÉ SIGNIFICA CADA PARÁMETRO

```python
from sklearn.ensemble import HistGradientBoostingClassifier

gb = HistGradientBoostingClassifier(
    learning_rate=0.05,        # 1
    max_iter=500,              # 2
    max_leaf_nodes=31,         # 3
    l2_regularization=1.0,     # 4
    early_stopping=True,       # 5
    random_state=42,
)
gb.fit(X_train, y_train)
gb.n_iter_                     # 6
```

1 **`learning_rate (η)`**
Cuánto de cada árbol nuevo se suma. ↓ aprende despacio y generaliza mejor (necesita más árboles) · ↑ rápido pero sobreajusta.

2 **`max_iter`**
Número máximo de árboles (rondas). Con early_stopping es un tope, no el valor final.

3 **`max_leaf_nodes`**
Tamaño de cada árbol. Árboles pequeños (8–31 hojas) = aprendices débiles, que es la idea.

4 **`l2_regularization`**
Freno sobre los valores de las hojas, como el α de Ridge. ↑ más conservador.

5 **`early_stopping`**
Separa una parte de train, mide cada ronda y para cuando deja de mejorar. Evita elegir max_iter a ojo.

6 **`n_iter_`**
Cuántos árboles usó realmente. Si es igual a max_iter, el tope estaba corto.

3 CUÁNDO USARLO Y QUÉ VIGILAR

VENTAJAS

- ✔ Precisión de punta en tablas (es lo que gana en Kaggle).
- ✔ Maneja nulos y categóricas sin preprocesar.
- ✔ Rápido gracias a los histogramas (Hist…).

CUIDADO CON

- ! Muchos hiperparámetros acoplados: η y max_iter se mueven juntos.
- ! Sobreajusta si lo dejas correr sin parada temprana.
- ! Caja negra: necesita SHAP para explicar una predicción.

DENTRO DEL `.FIT()`

QUÉ MINIMIZA	Log-loss (clasificación) o suma de cuadrados (regresión)
MOVIENDO QUÉ	Árboles pequeños añadidos en fila
CÓMO LO BUSCA	Descenso de gradiente en el espacio de árboles: cada árbol apunta al residual
PERILLA DEL OPTIMIZADOR	learning_rate, número de árboles (parada temprana), tamaño de cada árbol

EN EL CURSO Bank Marketing (S11, modelo de referencia de E4). Se explica con SHAP y se guarda en un Pipeline con joblib.

MÓDULO 4 · S10 · EVALUACIÓN · PARA CUALQUIER MODELO SUPERVISADO

Validación cruzada

Técnica de evaluación

`sklearn · cross_val_score · StratifiedKFold`

Propósito. Medir qué tan bien generaliza un modelo repitiendo el experimento train/test varias veces con particiones distintas, para no depender de un solo split con suerte.

FUNDAMENTO MÍNIMO

$CV = (1/K)\ \Sigma_k\ score_k$ · reporta media ± desviación

La fórmula está para leer las figuras, no al revés. El detalle completo vive en la celda A′ del notebook.

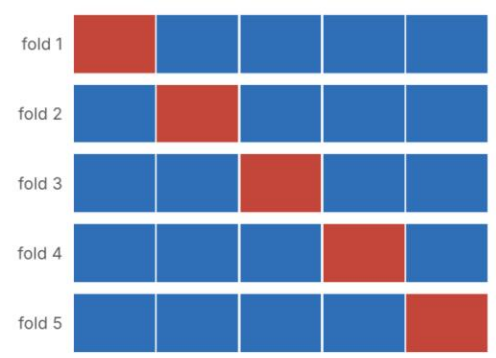
1 CÓMO FUNCIONA — LA HISTORIA EN CUATRO PASOS

1 · Un solo split: ¿tuve suerte?



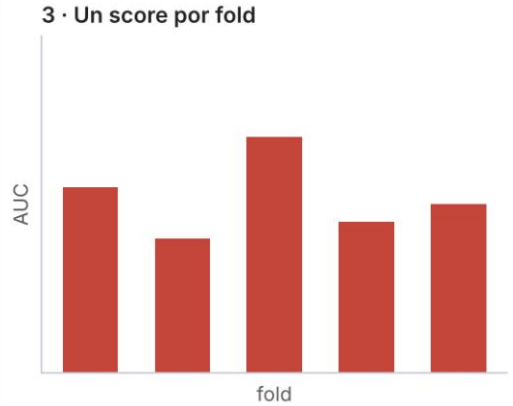
1. Un único split 80/20 da un número. ¿Fue representativo o tocaron justo los casos fáciles en test?

2 · 5 folds: cada parte es test una vez



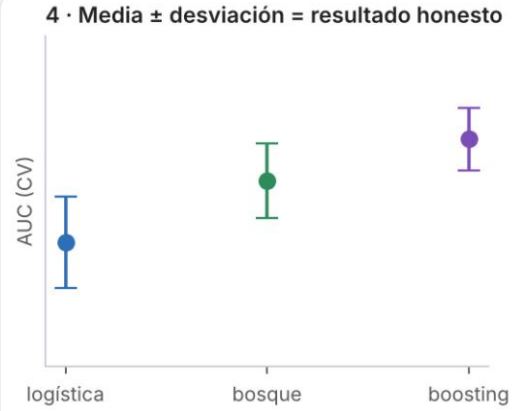
2. K = 5 folds: los datos se parten en 5. Cada fold es test una vez y train las otras cuatro. Todo punto se evalúa exactamente una vez.

3 · Un score por fold



3. Salen 5 scores. Su dispersión es información: dice cuánto cambia el resultado según qué filas tocaron.

4 · Media ± desviación = resultado honesto



4. La media ± desviación es el número honesto para comparar modelos. Dos modelos cuyas barras se cruzan no son «distintos».

2 ASÍ SE VE EN PYTHON · Y QUÉ SIGNIFICA CADA PARÁMETRO

```python
from sklearn.model_selection import (
    cross_val_score, StratifiedKFold)
from sklearn.pipeline import make_pipeline

pipe = make_pipeline(StandardScaler(),                    # 1
                     LogisticRegression())
cv = StratifiedKFold(n_splits=5,                          # 2
                     shuffle=True, random_state=0)        # 3
scores = cross_val_score(pipe, X_train, y_train,
                         cv=cv, scoring='roc_auc')        # 4
scores.mean(), scores.std()                               # 5
```

1. **`Pipeline`**
   El escalado (o cualquier preprocesamiento) va DENTRO: así se ajusta solo con el train de cada fold. Fuera = leakage en CV (trampa S10).
2. **`n_splits (K)`**
   Número de folds. 5 o 10 es lo habitual. ↑ más datos de train por fold y más costo; ↓ más varianza del estimado.
3. **`shuffle · random_state`**
   Baraja antes de partir (importante si los datos vienen ordenados por fecha o clase). Stratified mantiene el % de «sí» en cada fold.
4. **`scoring`**
   La métrica que decidiste en M2: 'roc_auc', 'recall', 'f1'... Nunca 'accuracy' con clases desbalanceadas.
5. **`mean · std`**
   Lo que se reporta. El std es la barra de error: sin él, la comparación entre modelos es un adorno.

3 CUÁNDO USARLO Y QUÉ VIGILAR

VENTAJAS

- ✔ Usa todos los datos para evaluar y para entrenar.
- ✔ Entrega incertidumbre, no un número suelto.
- ✔ Es la base de toda búsqueda de hiperparámetros.

CUIDADO CON

- ! Cuesta K veces más que un split.
- ! Con series de tiempo hay que usar TimeSeriesSplit, no barajar.
- ! El test final sigue siendo sagrado: la CV se hace solo con train.

DENTRO DEL `.FIT()`

QUÉ MINIMIZA	— (evalúa, no entrena)
MOVIENDO QUÉ	—
CÓMO LO BUSCA	Repite fit/score K veces sobre particiones distintas
PERILLA DEL OPTIMIZADOR	K, barajado, estratificación

EN EL CURSO Bank Marketing (S10). Trampa: preprocesamiento fuera del Pipeline = leakage en CV. Cierra el arco abierto en S3.

MÓDULO 4 · S11 · OPTIMIZACIÓN · PARA CUALQUIER MODELO

Búsqueda de hiperparámetros

Técnica de optimización

`sklearn · GridSearchCV · RandomizedSearchCV · Optuna`

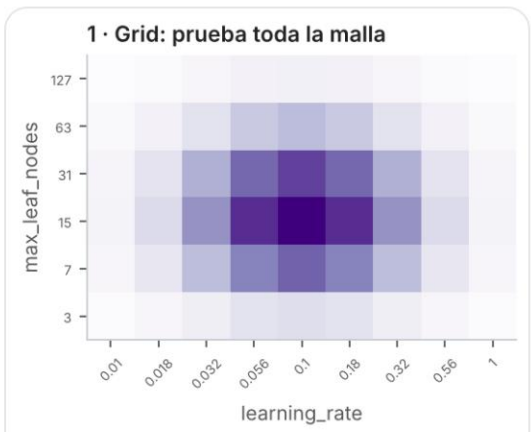
Propósito. Elegir los hiperparámetros del modelo probando combinaciones y midiendo cada una con validación cruzada. Es optimizar otra pérdida (la de validación) sin gradiente.

FUNDAMENTO MÍNIMO

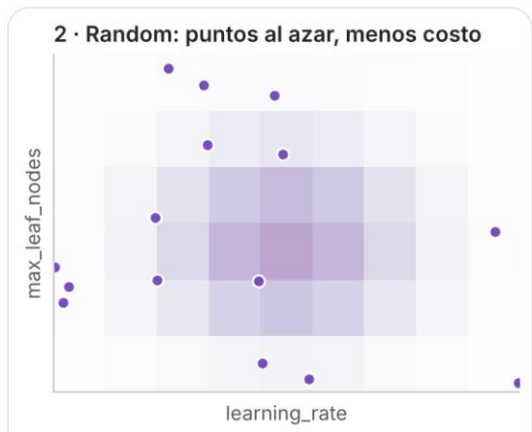
$\theta^* = \text{argmax}_\theta \; CV(\text{modelo}(\theta))$ · grilla, azar o bayesiana (TPE)

La fórmula está para leer las figuras, no al revés. El detalle completo vive en la celda A′ del notebook.

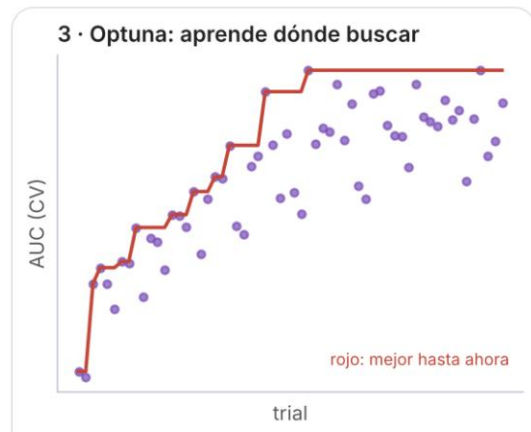
1 CÓMO FUNCIONA — LA HISTORIA EN CUATRO PASOS



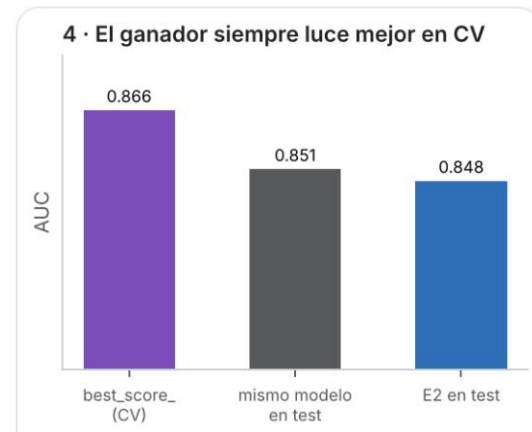
1 Grid: defines valores para cada hiperparámetro y pruebas TODAS las combinaciones (9 × 6 = 54 modelos × 5 folds). Exhaustivo, pero crece explosivamente.



2 Random: muestreas n_iter combinaciones al azar de rangos continuos. Con el mismo presupuesto suele encontrar algo igual de bueno que la grilla.



3 Optuna (TPE): cada trial usa los anteriores para proponer dónde buscar. La línea roja es el mejor hasta ahora: sube rápido y se estabiliza.



4 Maldición del ganador: entre 60 configuraciones, la mejor en CV tuvo también algo de suerte. Su score en test siempre es un poco menor. Reporta el test, no best_score_.

2 ASÍ SE VE EN PYTHON · Y QUÉ SIGNIFICA CADA PARÁMETRO

```python
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import loguniform, randint

espacio = {'gb__learning_rate': loguniform(0.01, 0.3),          # 1
           'gb__max_leaf_nodes': randint(8, 64)}
busq = RandomizedSearchCV(pipe, espacio,
    n_iter=60,                                                  # 2
    cv=cv, scoring='roc_auc',                                   # 3
    random_state=0, n_jobs=-1)
busq.fit(X_train, y_train)
busq.best_params_, busq.best_score_                             # 4
busq.best_estimator_.score(X_test, y_test)                      # 5
```

1 **espacio / param_distributions**
Qué perillas y en qué rangos. loguniform para tasas (0,01–0,3 vive en escala log); randint para enteros. El prefijo gb__ apunta al paso del Pipeline.

2 **n_iter / n_trials**
Presupuesto: cuántas configuraciones probar. Cada una cuesta K ajustes. En Optuna es n_trials en study.optimize().

3 **cv · scoring**
La misma CV y métrica de S10. Buscar con accuracy en Bank Marketing elegiría el modelo tonto.

4 **best_params_ · best_score_**
La configuración ganadora y su CV. best_score_ es optimista: es el máximo de 60 números con ruido.

5 **best_estimator_ en test**
El número que se reporta en E4. Se compara contra el baseline y contra E2, una sola vez.

3 CUÁNDO USARLO Y QUÉ VIGILAR

VENTAJAS

- ✔ Convierte el «tanteo» en un procedimiento reproducible.
- ✔ Random y Optuna encuentran buenos valores con pocos intentos.
- ✔ Con Pipeline, busca también opciones de preprocesamiento.

CUIDADO CON

- ! Costo = combinaciones × folds: puede ser horas.
- ! best_score_ exagera; el test puede decepcionar.
- ! Casi nunca es donde está la ganancia grande: primero datos, fugas y métrica.

DENTRO DEL `.FIT()`

QUÉ MINIMIZA	Métrica de validación cruzada (p. ej. AUC)
MOVIENDO QUÉ	Los hiperparámetros del modelo (η, hojas, α…)
CÓMO LO BUSCA	Grilla exhaustiva · azar · bayesiana (TPE, Optuna): optimización de caja negra
PERILLA DEL OPTIMIZADOR	Tamaño de la malla, n_iter, n_trials

EN EL CURSO Bank Marketing (S11, E4). Trampa: la maldición del ganador — creerle a best_score_.